

INTRODUCCIÓN A LA FORENSIA DIGITAL

VOLVIENDO A LOS DISCOS

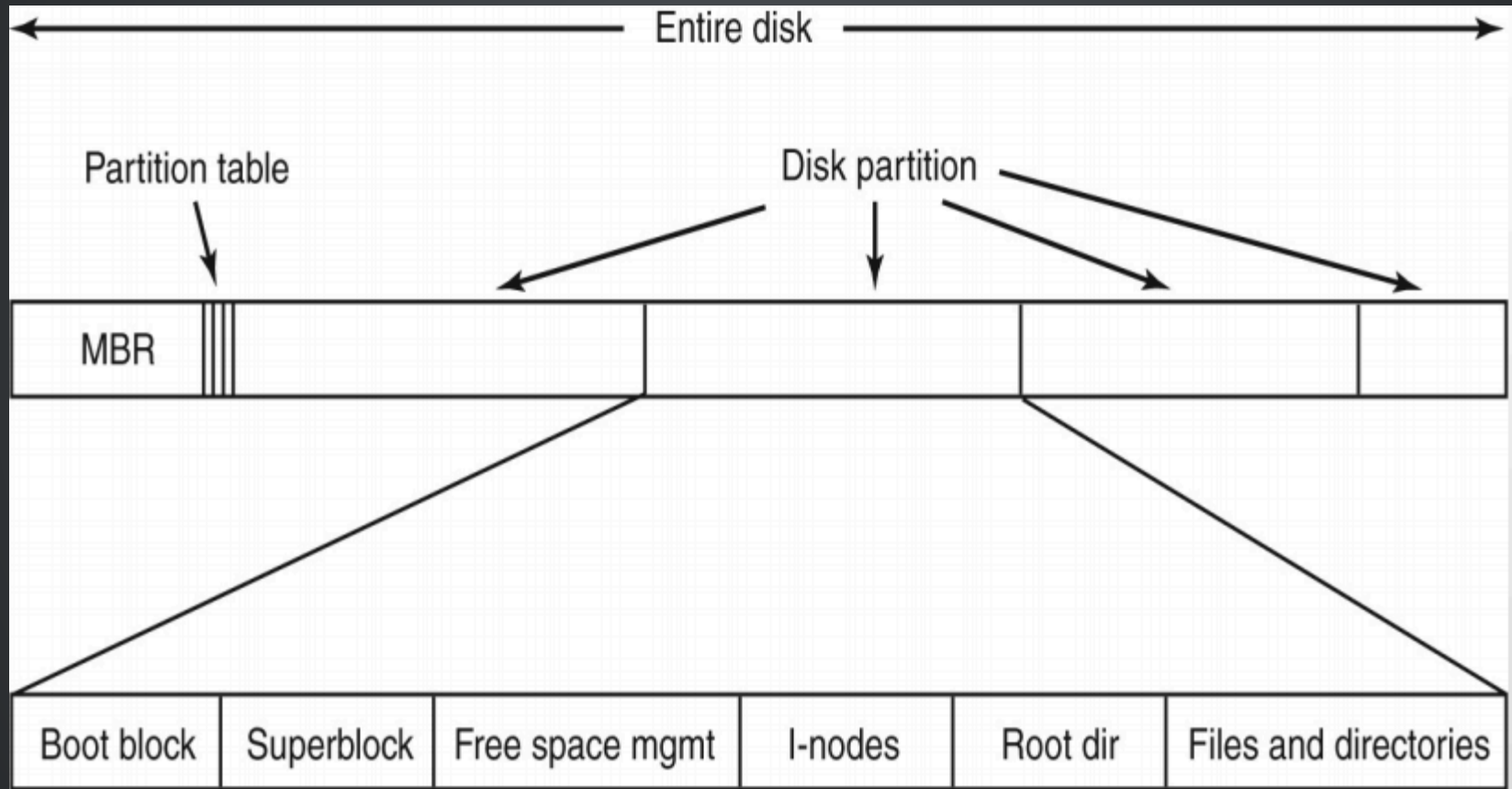


- 1956 IBM 305 RAMAC 4,2 megas, 1 tonelada, 3200u\$s mes
- 2019 sd de 256G 1.47 x 1.10 x 0.10 cm

PARTICIONES

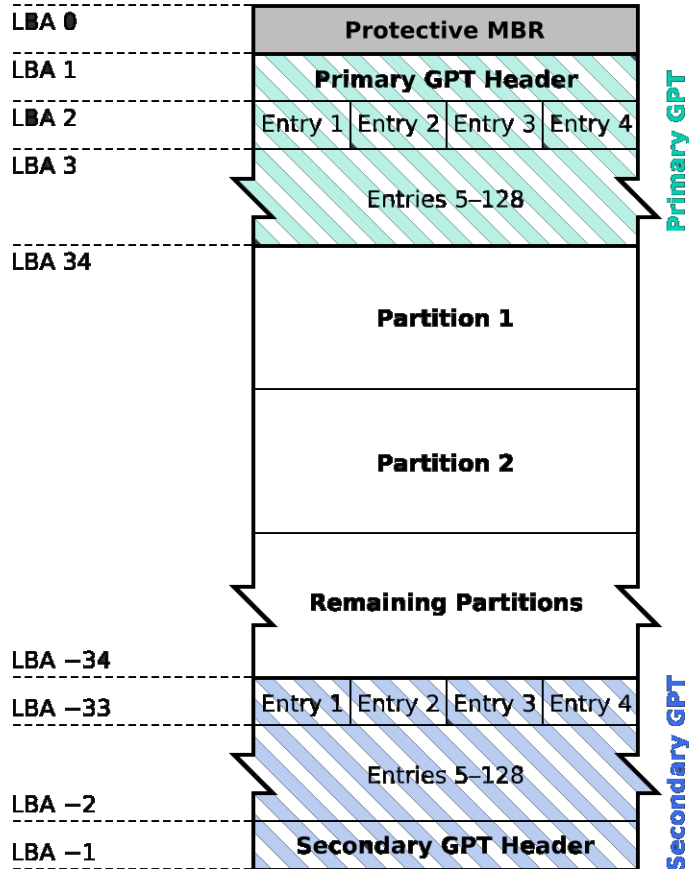
- Una partición de un disco duro es una división lógica en una unidad de almacenamiento, por ejemplo un disco duro o unidad flash.
- En ellas se alojan y organizan los archivos mediante un sistema de archivos.
- Existen distintos esquemas de particiones para la distribución de particiones en un disco. Los más conocidos y difundidos son MBR (Master Boot Record) y GPT (GUID Partition Table).
- Los discos ópticos (DVD, CD) utilizan otro tipo de particiones llamada UDF
- En los sistemas operativos se montan las particiones para poder accederlas:
 - En Windows generalmente **C:** o **D:**
 - En Linux en algún lugar del árbol de directorios, por ejemplo / o /home

GRÁFICAMENTE MBR



GRÁFICAMENTE GPT

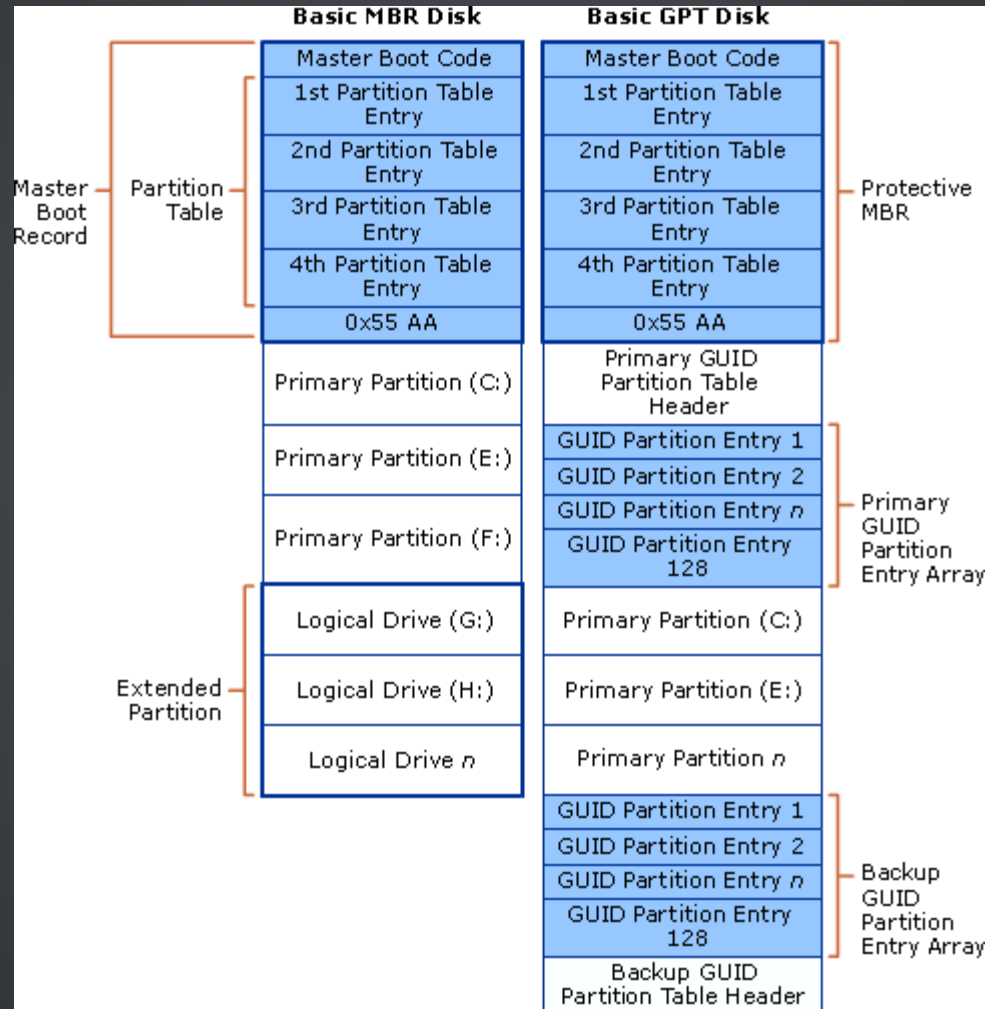
GUID Partition Table Scheme



Formato de entrada de partición GUID

Offset	Longitud	Contenido
0 (0x00)	16 bytes	Tipo de partición GUID
16 (0x10)	16 bytes	GUID único de partición
32 (0x20)	8 bytes	Inicio LBA (little endian)
40 (0x28)	8 bytes	Fin LBA (inclusive, generalmente impar)
48 (0x30)	8 bytes	Indicadores (p.ej. bit 60 denota sólo lectura)
56 (0x38)	72 bytes	Nombre de partición (36 caracteres UTF-16LE)
	128 bytes totales	

COMPARÁNDOLAS



COMPARÁNDOLAS

- Limitaciones de MBR:
 - Cantidad de particiones 4 primarias
 - Tamaño de las particiones de hasta 2TB
 - Tabla de particiones sin backup
- Limitaciones de GPT:
 - Algunos sistemas operativos no lo reconocen y no lo pueden usar
 - Dependencia de UEFI

SISTEMAS DE ARCHIVOS

- Cuando trabajamos en procesos forenses en dispositivos de almacenamiento hay un par de cuestiones a las que tenemos que prestar atención antes de proceder:
 - ¿Cómo guarda los datos? -> es decir el contenido de los archivos
 - ¿Cómo guarda los metadatos? -> por ejemplo nombre, tamaño, etc
 - ¿Cómo ocurren los cambios en el disco cuando archivos y directorios son eliminados o modificados?
- Revisaremos los más comunes:
 - FAT y NTFS
 - Ext

FILE ALLOCATION TABLE(FAT)

- Fue le primer sistema de archivos inventado por Microsoft
- En DOS se utilizaban las versiones FAT12 y FAT16
- Desde que apareció Windows y hasta el día de hoy se suele utilizar el FAT32.
- Es ampliamente utilizado por muchos dispositivos pequeños, por ejemplo, grabadores de voz, cámaras o consolas de juegos.

ESTRUCTURA FAT32

- Sector de booteo
- Sector de Información del filesystem
- Tablas FAT
- Datos de Archivos y subdirectorios



BOOT SECTOR

- Contiene código ejecutable y es al que el MBR pasa el control al iniciar el equipo
- Contiene además algunos datos relevantes:
 - Sector Size (casi siempre 512 bytes)
 - Sectores por cluster
 - Cantidad de tablas FAT
 - Cantidad total de sectores
- Sólo en FAT32
 - Sectores por FAT
 - Label del volumen

FILE SYSTEM INFORMATION SECTOR

- Sólo en FAT32
- Se utiliza para saber cuanto espacio libre queda y donde está localizado

DOS TIPOS DE CLUSTERS

- **Directory Clusters:** contienen la estructura de los directorios
- **Data clusters:** contienen el contenido de los archivos

DIRECTORIOS (O DIRECTORY ENTRIES)

- File system metadata
- Nombre de los archivos, timestamp e inicio del cluster de datos para los archivos
- Solo se usan para las entradas de directorios

ROOT DIRECTORY

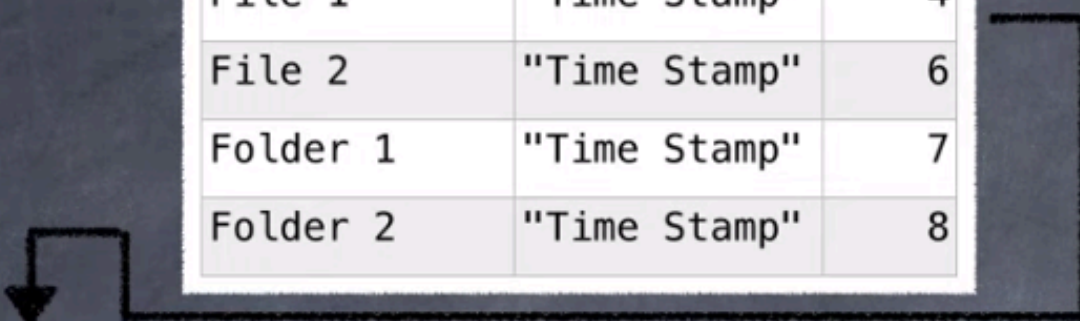
- Es el primer directory cluster del disco
- Contiene una entrada para cada archivo o directorio almacenado en la raíz del filesystem
- Contienen todos los metadatos
 - Nombre
 - Tamaño
 - Cluster Inicial
 - Atributos
 - MAC times
 - Create time
 - Access Time
 - Modified Time
- Esta información cambia cada vez q un archivo es creado o modificado.

CLUSTERS

File 1	"Time Stamp"	4
File 2	"Time Stamp"	6
Folder 1	"Time Stamp"	7
Folder 2	"Time Stamp"	8

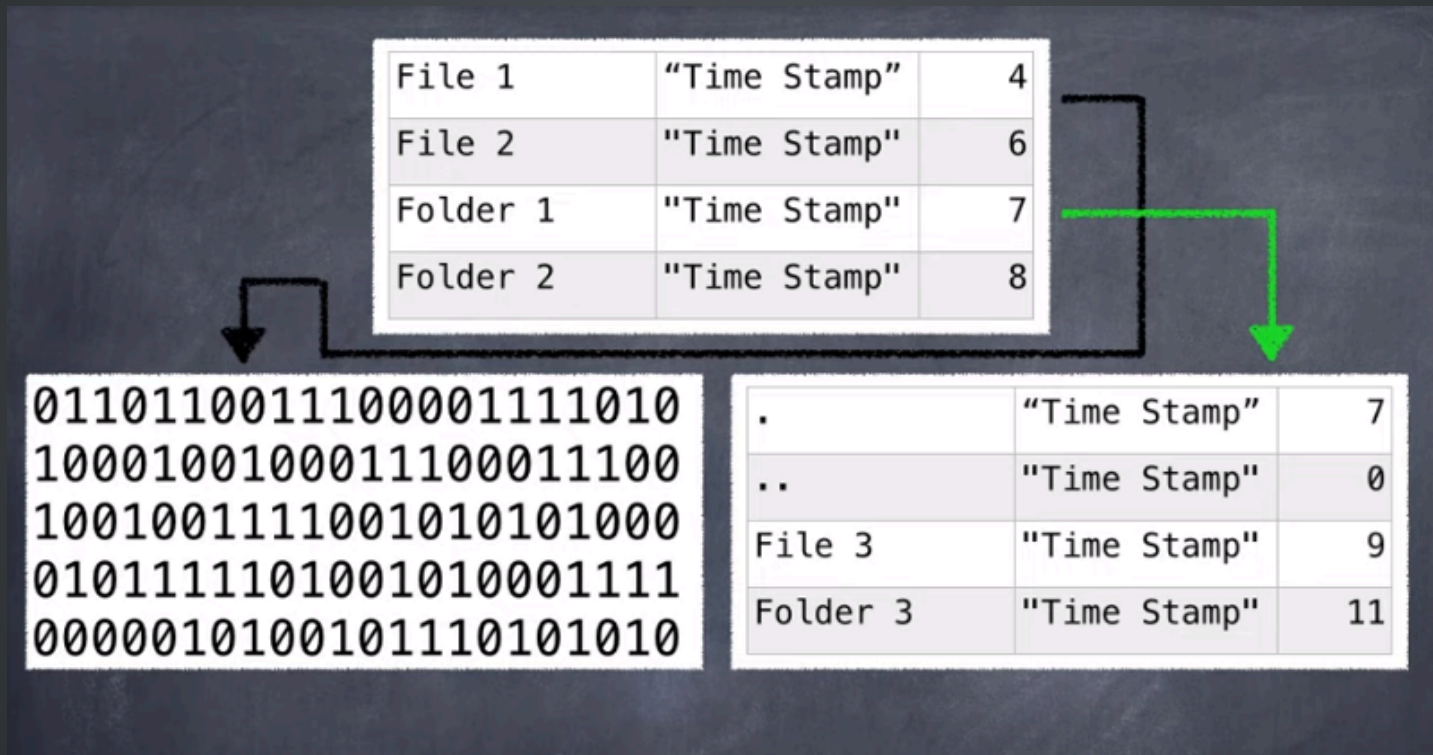
CLUSTERS

File 1	"Time Stamp"	4
File 2	"Time Stamp"	6
Folder 1	"Time Stamp"	7
Folder 2	"Time Stamp"	8

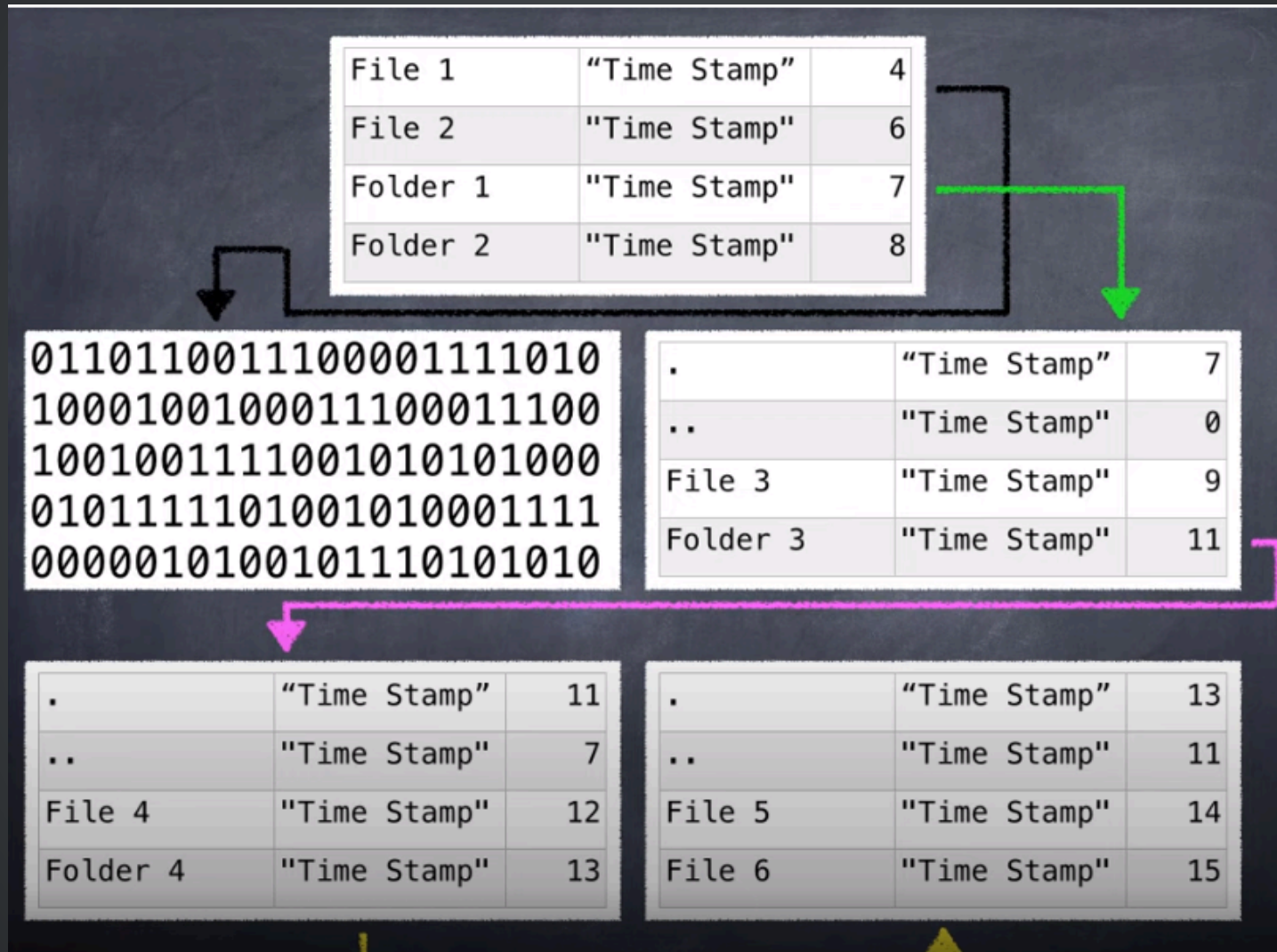


0110110011100001111010
1000100100011100011100
1001001111001010101000
0101111101001010001111
0000010100101110101010

CLUSTERS



CLUSTERS



TABLAS FAT

- Se utilizan para relacionar la cadena de clusters que contienen los datos de una carpeta o archivo.
 - Un Cluster es una serie contigua de sectores alocados.
- Los índices de las cadenas no se almacenan de manera contigua, sino que FAT utiliza un tipo de lista de links y un marcador EOF (fin de archivo) para indicar donde termina la cadena
- La tarea principal es saber el estado de alocaación de los cluster:
 - Alocado (junto con la dirección del siguiente cluster asociado al archivo)
 - No alocado
 - Fin de archivo
 - Sector defectuoso.
- La FAT2 es una copia de la FAT1.

DATA AREA

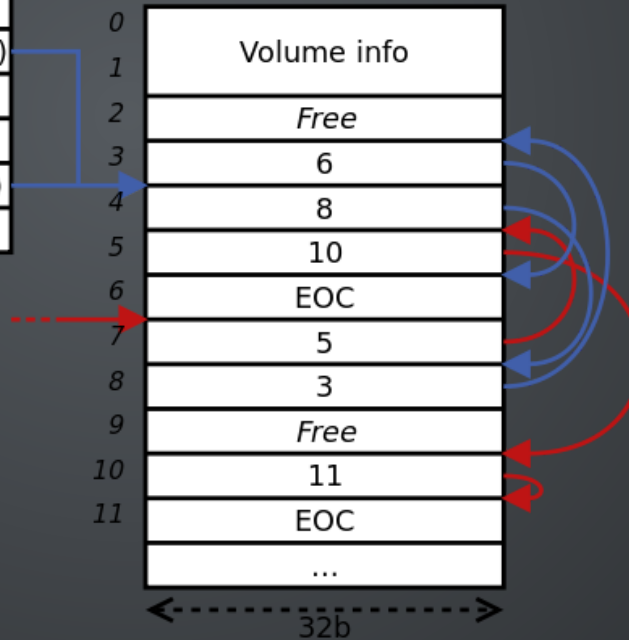
- Al Boot Record, las tablas FATs, y el Root Directory en conjunto se los suele llamar el System Area.
- Al resto del espacio en el disco es lo que llamamos Data Area, y es donde el contenido de los archivos están realmente almacenados.
- Cuando un archivo es eliminado por el sistema operativo, los datos almacenados aquí permanecen intactos hasta que se los sobrescriba.
- En el área de datos encontraremos data clusters y directory clusters.

EJEMPLO DE ARCHIVO

Directory table entry (32B)

Filename (8B)
Extension (3B)
Attributes (1B)
Reserved (1B)
Create time (3B)
Create date (2B)
Last access date (2B)
First cluster # (MSB, 2B)
Last mod. time (2B)
Last mod. date (2B)
First cluster # (LSB, 2B)
File size (4B)

File allocation table



FAT'S ROOT DIRECTORY

- Se requiere un método especial para encontrar la raíz del filesystem
- Contiene la información de archivos y subdirectorios que están en la raíz del filesystem
- Una vez que encontramos la raíz el resto se puede encontrar mediante recursión sobre los directorios en los contenidos de los mismos
- En FAT es posible encontrarla: ya que aparece inmediatamente luego de la última copia de la FAT, con sólo saber el tamaño de la FAT y la cantidad de copias se encuentra: $(\text{size of FAT}) * (\text{number of FATs}) + 1$

GRÁFICAMENTE:

File Allocation Table

File.txt



is

Block 2



Block 6



Block 3



Block 5



	<u>FAT</u>	
	Busy	Next
0	0	
1	1	-1
2	1	6
3	1	5
4	1	-1
5	1	-1
6	1	3
7		

Directory Table Former

filename	starting block	meta data
foo	1	

/foo

filename	starting block	meta data
File.txt	2	

LECTURA EN FAT

- Suponga que queremos leer un archivo en "usuarios/pepe/nota.txt"
- Pasos:
 - Encontrar el root directory
 - Buscar la entrada para "usuarios" y obtener el cluster
 - Repetir el paso para encontrar el cluster de la entrada para "pepe" dentro de "usuarios"
 - Encontrar la entrada para "nota.txt"
 - Recorrer paso a paso la cadena de clusters

BORRANDO UN ARCHIVO

- La entradas en la tabla FAT se setean a 0
- La primera letra de los nombre de los archivos se setea a 0xe5
- La entrada de directorio se marca como libre

RECUPERAR EN FAT

- Excepto por el primer carácter se pueden recuperar los nombres de los archivos
- Los metadatos de los archivos se pueden recuperar
- Los contenidos son más difíciles de recuperar:
 - Recordar que las cadenas en la tabla FAT se marcaron a 0
 - Pero igual tenemos un puntero al primer cluster
 - A partir del primer cluster se puede leer el tamaño del archivo y esperar que los clusters no hayan sido sobrescritos.

RESUMIENDO FAT

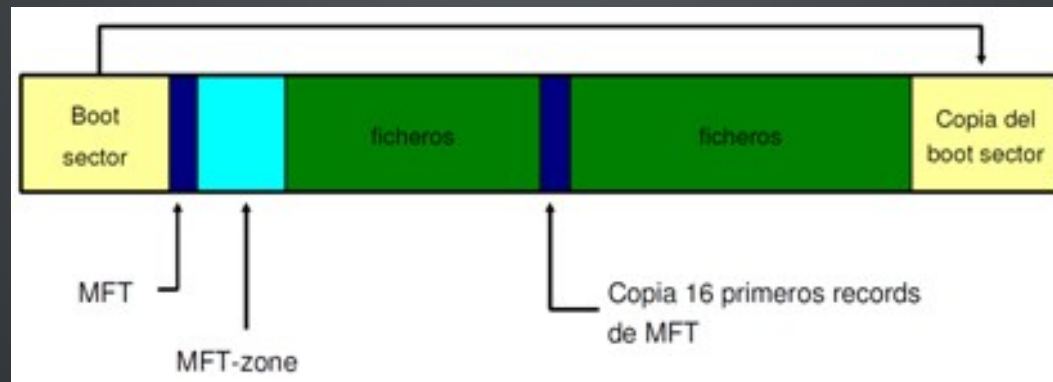
- Muy utilizado aún hoy en día, sobre todo para dispositivos como cámaras, consolas de juegos o grabadores.
- Soportado por todos los sistemas operativos actuales, no pasa lo mismo por ejemplo con NTFS o Ext*
- Al ser tan popular entender su estructura y limitaciones es un "debe" para cualquier investigador forense.

NTFS

- Este es el sistema de archivos de todos los sistemas Windows modernos
- Al ser Windows el sistema operativo más difundido, es bastante probable que el investigador se encuentre con el bastante habitualmente en el día a día.
- Si lo comparamos con FAT veremos que es mucho más maduro y avanzado, aunque conserva la idea de listas enlazadas.
- Más Info sobre NTFS: [https://technet.microsoft.com/en-us/library/cc781134\(WS.10\).aspx](https://technet.microsoft.com/en-us/library/cc781134(WS.10).aspx)

ESTRUCTURA NTFS

- Todo es considerado un archivo, incluso los metadatos.
- Todos los datos están almacenados en la Master File Table
- La locación y el tamaño del MFT se almacena en la primer entrada del MFT
- Dos copias de la MFT son almacenadas en caso de corrupción. Si la primer copia se corrompe NTFS leerá de la segunda copia.



FORMATO DE ENTRADA NTFS

- La MFT es una lista de Records que contienen atributos:
 - Records:
 - Estructura que guarda información de un archivo relacionando atributos
 - Normalmente 1 registro por archivo, aunque a veces se necesitan más
 - Atributos:
 - Información relacionada a un archivo.
 - Pueden ser desde bits, por ejemplo para los permisos de escritura y lectura, hasta gigabytes cuando es un archivo de video
 - No hay distinción entre los datos de un archivo y los atributos que lo describen

LA ENTRADA INICIAL DEL MFT

- Las primeras 16 entradas del MFT están reservadas por el sistema operativo para guardar metadatos
- Estos archivos se ocultan a simple vista, pero pueden accederse utilizando técnicas forenses.
- La primer entrada (la 0) es la propia MFT

MFT AND METADATA FILES

Metadata Files

Files 0 – 15 are reserved for metadata files in the MFT, usually only the first 12 are used by MS.

0	\$Mft – MFT	8	\$BadClus – Bad-cluster file
1	\$MftMirr – MFT Mirror	9	\$Secure – Security settings file
2	\$LogFile – Log File	10	\$UpCase – Uppercase character mapping
3	\$Volume – Volume File	11	\$Extend – Extended metadata directory
4	\$AttrDef – Attribute definition table	12	Unused
5	\ - Root directory	13	Unused
6	\$Bitmap - Voume cluster allocation file	14	Unused
7	\$Boot – Boot sector	15	Unused

NTFS FILE RECORD ATTRIBUTES

Attribute Type	Description
Standard Information	Information such as access mode (read-only, read/write, and so forth) timestamp, and link count.
Attribute List	Locations of all attribute records that do not fit in the MFT record.
File Name	A repeatable attribute for both long and short file names. The long name of the file can be up to 255 Unicode characters. The short name is the 8.3, case-insensitive name for the file. Additional names, or hard links, required by POSIX can be included as additional file name attributes.
Data	File data. NTFS supports multiple data attributes per file. Each file typically has one unnamed data attribute. A file can also have one or more named data attributes.
Object ID	A volume-unique file identifier. Used by the distributed link tracking service. Not all files have object identifiers.
Logged Tool Stream	Similar to a data stream, but operations are logged to the NTFS log file just like NTFS metadata changes. This attribute is used by EFS.
Reparse Point	Used for mounted drives. This is also used by Installable File System (IFS) filter drivers to mark certain files as special to that driver.
Index Root	Used to implement folders and other indexes.
Index Allocation	Used to implement the B-tree structure for large folders and other large indexes.
Bitmap	Used to implement the B-tree structure for large folders and other large indexes.
Volume Information	Used only in the \$Volume system file. Contains the volume version.

MFT RECORD CHEATSHEET



MASTER FILE TABLE RECORD

> Invoke-IR

BY: JARED ATKINSON
TEMPLATE BY: ANGE ALBERTINI



```
000 46 49 4c 45 30 00 03 00 08 9c 73 8a 00 00 00 00
010 c5 01 02 00 38 00 01 00 b8 01 00 00 00 04 00 00
020 00 00 00 00 00 00 00 00 04 00 00 00 ea 53 00 00
030 2e 5c 00 00 00 00 00 00 10 00 00 00 60 00 00 00
040 00 00 00 00 00 00 00 00 48 00 00 00 18 00 00 00
050 1d 8e 30 3d ae 99 d0 01 48 e8 ba 65 e9 9b d0 01
060 48 e8 ba 65 e9 9b d0 01 1d 8e 30 3d ae 99 d0 01
070 20 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
080 00 00 00 00 ad 05 00 00 00 00 00 00 00 00 00 00
090 f0 95 e5 13 00 00 00 00 30 00 00 00 78 00 00 00
0A0 00 00 00 00 00 00 00 03 00 5a 00 00 00 18 00 01 00
0B0 85 ec 02 00 00 00 38 00 1d 8e 30 3d ae 99 d0 01
0C0 1d 8e 30 3d ae 99 d0 01 1d 8e 30 3d ae 99 d0 01
0D0 1d 8e 30 3d ae 99 d0 01 00 00 00 00 00 00 00 00
0E0 00 00 00 00 00 00 00 00 20 00 00 00 00 00 00 00
0F0 0c 02 48 00 45 00 4c 00 4c 00 4f 00 57 00 7e 00
100 31 00 2e 00 54 00 58 00 54 00 78 00 74 00 00 00
110 30 00 00 00 78 00 00 00 00 00 00 00 00 00 02 00
120 5e 00 00 00 18 00 01 00 85 ec 02 00 00 00 3b 00
130 1d 8e 30 3d ae 99 d0 01 1d 8e 30 3d ae 99 d0 01
140 1d 8e 30 3d ae 99 d0 01 1d 8e 30 3d ae 99 d0 01
150 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
160 20 00 00 00 00 00 00 00 0e 01 68 00 65 00 6c 00
170 6c 00 6f 00 77 00 6f 00 72 00 6c 00 64 00 2e 00
180 74 00 78 00 74 00 00 80 00 00 00 28 00 00 00 00
190 00 00 18 00 00 01 00 0e 00 00 00 18 00 00 00 00
1A0 ff ff 74 00 65 00 73 00 74 00 00 00 0a 00 00 00
1B0 ff ff ff 82 79 47 11 00 00 00 00 00 00 00 00 00
1C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
1F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 2e 5c
200 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
210 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
220 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
230 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
240 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
250 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
260 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
270 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
280 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
290 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
2F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
300 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
310 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
320 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
330 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
340 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
350 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
360 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
370 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
380 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
390 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3A0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3B0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3C0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3D0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3E0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
3F0 00 00 00 00 00 00 00 00 00 00 00 00 00 00 2e 5c
```

FILE RECORD HEADER

ATTRIBUTES

\$STANDARD_INFORMATION ATTRIBUTE

\$FILE_NAME ATTRIBUTE

\$FILE_NAME ATTRIBUTE

\$DATA ATTRIBUTE

FLAGS

0X01 - IN USE
0X02 - DIRECTORY

FIELDS

magic
offset to us
size of us
logical sequence number
sequence number
hardlinks
offset to attributes
flags
real size
allocated size
reference to base
next attribute id
alignment bytes
record numbers
update sequence
update sequence array

VALUES-

FILE
0x30
0x03
8A739C08
0x1C5
0x02
0x38
0x01
0x1B8
0x400
0x0000000000000000
0x04
0x00
0x53EA
0x02 0x00
0x00 0x00 0x00 0x00

REAL VS ALLOCATED SIZE

Allocated Size - Size of allocated disk space. This size will be divisible by the size of a disk cluster.

Real Size - Actual size of file contents. This size is the one referenced by the "dir" command.

If real and allocated size are 0, then the file's contents are contained within a resident data attribute in the file's MFT record.

SOBRE LOS DIRECTORIOS

- Los directorios se guardan en un árbol B
- El atributo **\$INDEX_ROOT** contiene información de los archivos y subdirectorios del directorio actual.
- Si el directorio es grande, se utilizan también las entradas **\$INDEX_ALLOCATION** y **\$BITMAP**.

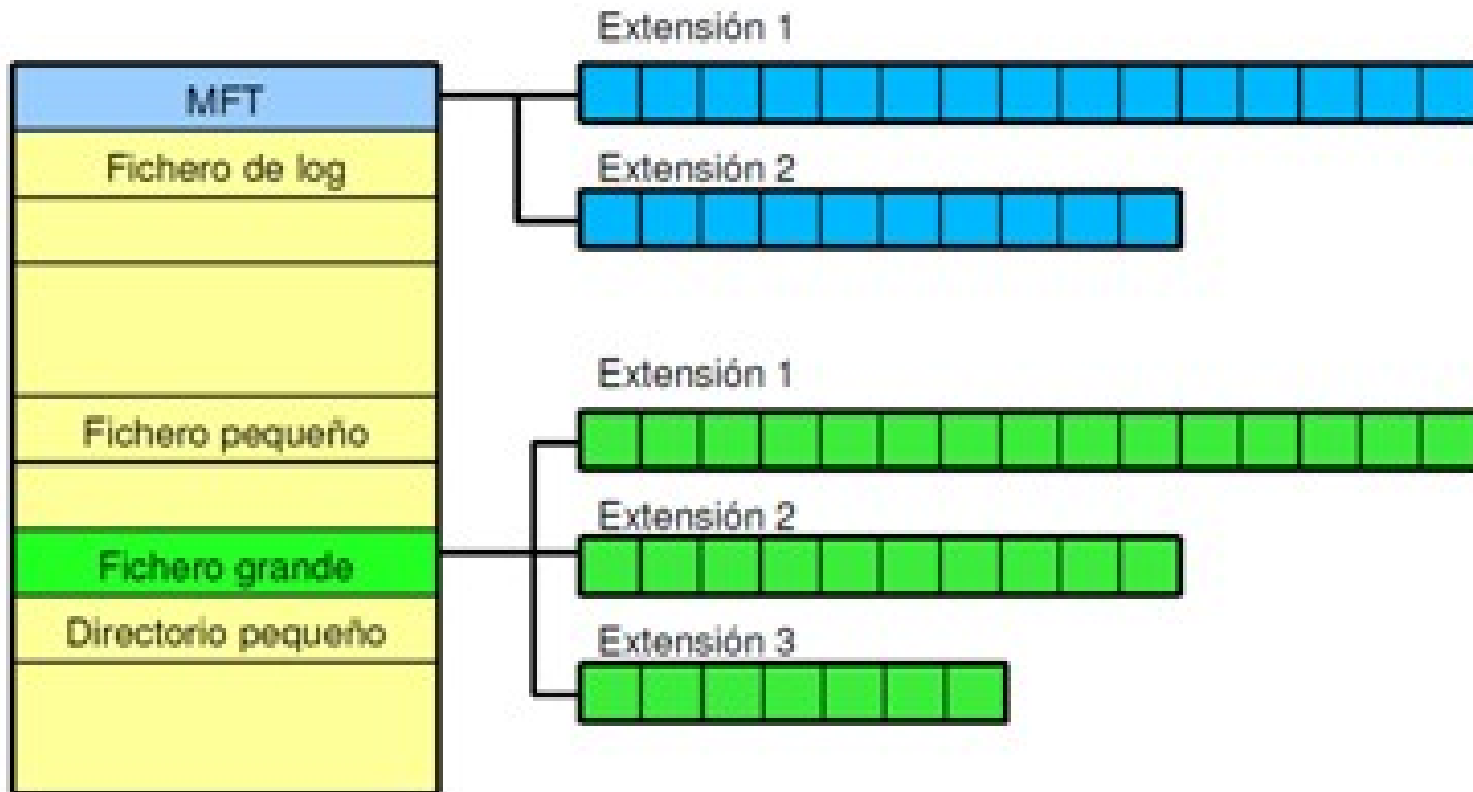
SOBRE LOS ARCHIVOS-ATRIBUTOS IMPORTANTES

- Los archivos tienen sus propias entradas en la MFT y el contenido se almacena en **\$DATA**
- MAC times de archivos y carpetas, y M time de las entradas de la MFT se guardan en **\$STANDARD_INFORMATION**
- En **\$FILE_NAME** se guarda el nombre del archivo y el puntero a la entrada MFT del directorio padre, incluyendo MAC times. Esto se utiliza para recuperar los archivos.

FORMATO DE ENTRADA NTFS

- Hay 2 tipos de Records que dependen del tamaño del archivo
 - Si es menor a 1500 bytes:
 - Record con atributos residentes
 - Incluyen los datos del fichero
 - Optimizan el acceso a disco
 - Si es mayor a 1500 bytes:
 - Record con atributos NO residentes
 - Record con índices
 - Índices que apuntan a bloques de información
 - Si hay tantos índices que no caben en un record MFT se utiliza un puntero que apunta a otro record de la MFT que contendrá índices.

RECORDS Y ATRIBUTOS MFT



MACE TIMESTAMPS:

- Las marcas de tiempo y hora que NTFS guarda sobre cada archivo se relacionan a:
 - M – modificado
 - A – accedido
 - C – creado
 - E – entrada modificada (también conocido como ‘B’ por Born)
- Windows no suele actualizar FILE_NAME sino que lo hace en el atributo STANDARD_INFORMATION, frecuentemente coincide cuando creó, movió o renombró el archivo

¿QUÉ SE ACTUALIZA?

\$FILE_NAME	Rename	Local Move	Volume Move	Copy	Access	Modify	Create	Delete
Modification		X	X	X			X	X
Accessed			X	X			X	
Change (meta)		X	X	X			X	X
Born			X	X			X	
\$STANDARD_INFO	Rename	Local Move	Volume Move	Copy	Access	Modify	Create	Delete
Modification						X	X	
Accessed			X	X	X	X	X	
Change (meta)	X	X	X	X			X	X
Born				X			X	

- Las filas representan los 4 tipos de timestamps y las columnas acciones realizadas desde la GUI de Windows.
- Cuando hay una X significa que ese valor se actualizó con esa acción.

CUANDO SE BORRA UN ARCHIVO

- La entrada del archivo en el MFT del directorio se elimina
- El flag in-use del MFT del archivo se limpia
- El atributo DATA de la entrada BITMAP para el archivo se limpia
- Los cluster del archivo se desalocan
- Se reordena el árbol B del directorio

¿CÓMO RECUPERAR UN ARCHIVO?

- Encontrar archivos borrados es simple, se recorre el MFT y se buscan entradas marcadas como not in-use
 - Si los archivos son pequeños (700 o menos bytes) se pueden recuperar completamente porque se alojan en la MFT
 - La recuperación de archivos más grandes depende si los clusters libres fueron o no reasignados luego de haberse eliminado el archivo

JOURNALING

- El journaling es un mecanismo por el cual el sistema de archivos puede implementar transacciones.
- Se basa en llevar un journal o registro de diario en el que se almacena la información necesaria para restablecer los datos afectados por la transacción en caso de que ésta falle.
- Esta idea se utiliza en los sistemas modernos para asegurar consistencia por ejemplo, ante cortes de energía eléctrica o fallos de disco.
- Antes de escribirse en el bloque del disco, primero se escribe en el journal del filesystem
- En el caso de una caída inesperada el journal se puede rehacer para asegurar la consistencia.

NTFS JOURNALING

- Se denomina logging
- El journal se almacena en la entrada 2 de la MFT, el \$LogFile
- Se puede analizar utilizando herramientas forenses para determinar cambios recientes en el filesystem
- De la misma manera que lo utilizan herramientas de recovery

ALGUNOS DETALLES

- Tener en cuenta que Vista y posteriores tiene los timestamp de acceso deshabilitados por defecto. Para habilitarlo hay que editar el registro clave
HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\FileSystem.
.
- Más info:
 - <https://support.microsoft.com/es-es/kb/299648>
 - <http://es.slideshare.net/ctin/time-stamp-analysis-of-windows-sys>

COMPARATIVA DE FS

http://www.ntfs.com/ntfs_vs_fat.htm

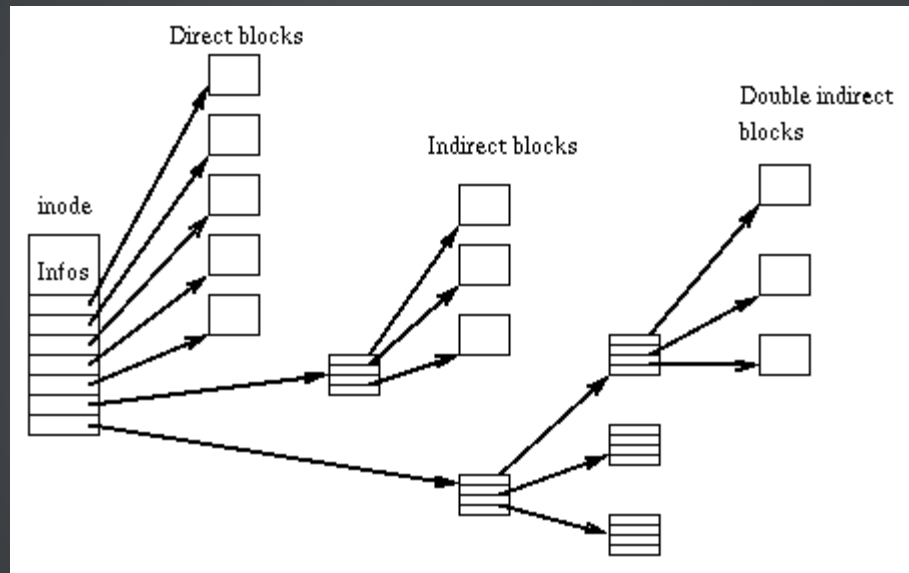
EXTENDED FILE SYSTEM

- El sistema de archivos extendido (extended file system o ext), fue el primer sistema de archivos creado específicamente para el sistema operativo Linux.
- Surge como reemplazo de MINIX
- Tuvo 3 actualizaciones:
 - Ext2, el segundo sistema de archivos extendido
 - Ext3, el tercero sistema de archivos extendido
 - Ext4, el cuarto sistema de archivos extendido

EXT2

- El sistema de ficheros tiene una tabla donde se almacenan los i-nodos.
- Este término proviene de "index node"
- Cada archivo o directorio es representado por un inodo
- El inodo incluye información de tamaño, permisos, propietario y lugar en el disco donde está el contenido.
- En cuanto a la ubicación, es una referencia a un sector del disco donde están todas y cada una de las referencias a los bloques del archivo fragmentado.
- El tamaño de los bloques se especifica cuando se crea el sistema de archivos.

ESTRUCTURA



EJEMPLO

- Buscar el i-nodo del archivo `/usr/ast/mbox`

Root directory

1	.
1	..
4	bin
7	dev
14	lib
9	etc
6	usr
8	tmp

Looking up
usr yields
i-node 6

I-node 6 is for /usr

Mode size times
132

I-node 6
says that
/usr is in
block 132

Block 132 is /usr directory

6	•
1	••
19	dick
30	erik
51	jim
26	ast
45	bal

/usr/ast
is i-node
26

I-node 26 is for /usr/ast

Mode size times
406

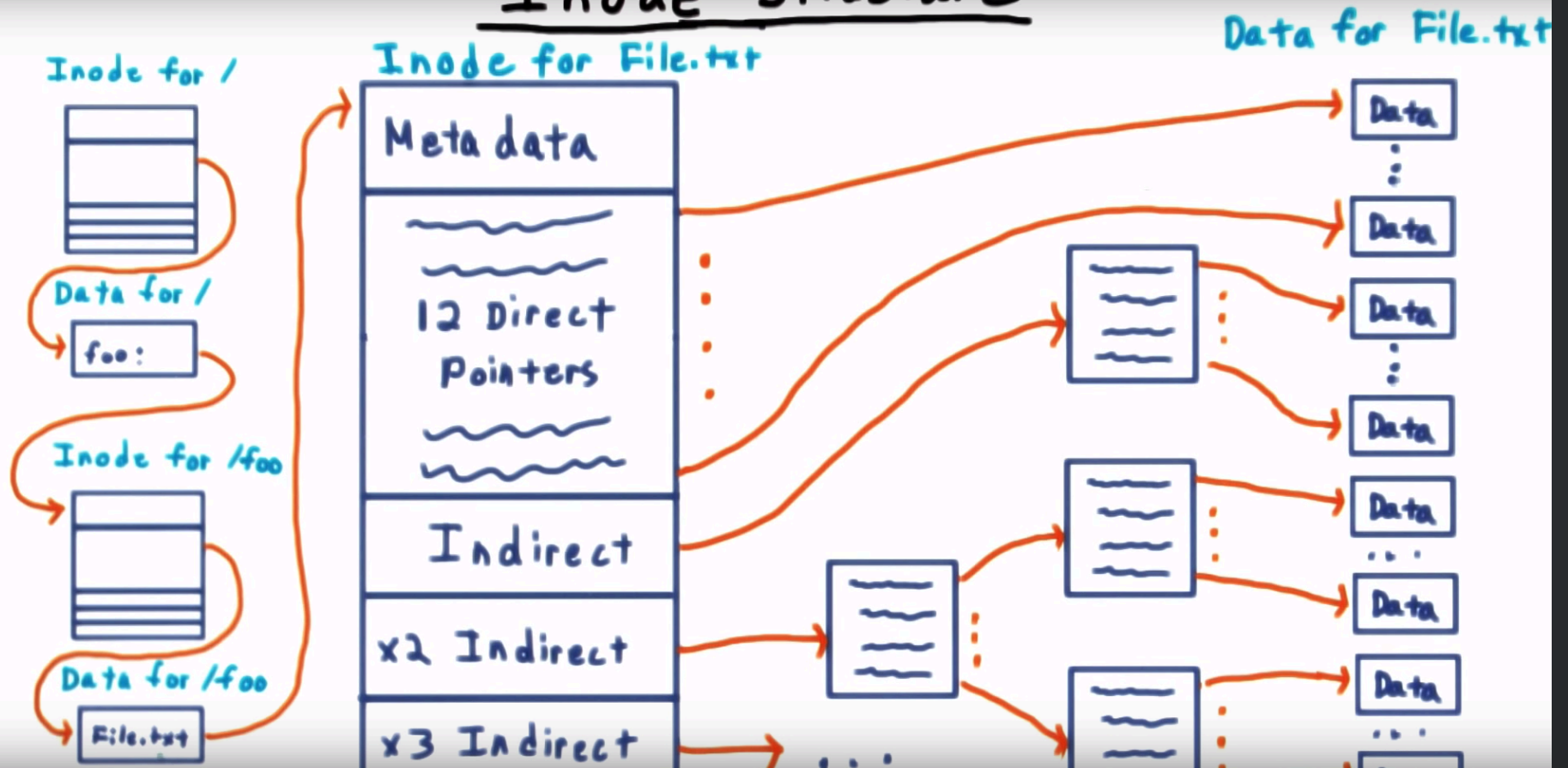
I-node 26
says that
/usr/ast is in
block 406

Block 406 is /usr/ast directory

26	•
6	••
64	grants
92	books
60	mbox
81	minix
17	src

/usr/ast/mbox
is i-node
60

Inode Structure



EXT3

- ext3 (third extended filesystem o "tercer sistema de archivos extendido") es un sistema de archivos con registro por diario (journaling).
- Fue el sistema de archivos más usado en distribuciones GNU/Linux, aunque en la actualidad ha sido remplazado por su sucesor, ext4.
- La principal diferencia con su predecesor ext2 es el journaling.

EXT3: DISPONIBILIDAD

- Tras un corte eléctrico o una caída inesperada del sistema (también se denomina cierre no limpio del sistema), se debe comprobar con el programa `e2fsck` cada sistema de archivos ext2 montado en la máquina para ver si es consistente.
- El proceso de comprobación lleva mucho tiempo y puede prolongar el tiempo de arranque del sistema de un modo significativo, especialmente si hay grandes volúmenes que contienen un elevado número de archivos.
- Durante este proceso, no se puede acceder a los datos de los volúmenes.

JOURNALING

- Recordemos: un sistema de archivos con journaling es un tipo de sistema que permite al SO mantener un registro del cambio de todos los archivos del sistema antes de escribir los datos en el disco.
- Este registro o log es llamado journaling y es usualmente un log circular que se ubica en un lugar especialmente alocado del sistema.
- La primera estructura en el journal es llamado journal superblock y mantiene información respecto al tamaño del bloque, numero total de bloques que el journal tiene disponibles para almacenar, donde empieza realmente el journal, número de secuencia de la primera transacción, donde esta alocada la primera transacción e información general del journal.

JOURNALING

- El mecanismo del journal mantiene un registro de los cambios en los archivos del sistema con el uso de secuencias de transacción.
- Las mismas se componen de:
 - **Descriptor del bloque:** toda transacción empieza con un bloque que describe el principio de la transacción.
 - **Bloque de metadatos:** Es donde los cambios son registrados. Puede haber uno o más en cada transacción.
 - **Bloque de commit:** Básicamente indica el fin de la transacción.
 - **Bloque de revocar:** Si hubo un error en la transacción un bloque de este tipo es creado y mantiene una lista de bloques del sistema que se necesitan restaurar cuando se haga el chequeo de consistencia.

JOURNALING



EXT3: DISPONIBILIDAD

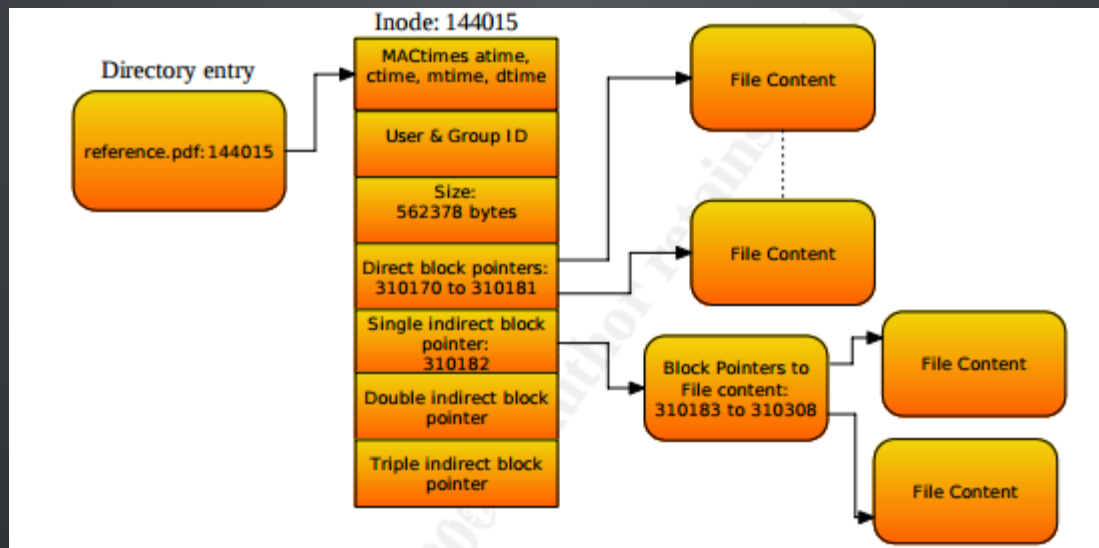
- Con journaling en ext3 ya no es necesario realizar este tipo de comprobación en el sistema de archivos después de un cierre no limpio del sistema.
- Únicamente se realiza una comprobación de consistencia en los casos puntuales en los que se producen determinados errores de hardware, como, por ejemplo, fallos en el disco.
- El tiempo empleado para recuperar un sistema de archivos ext3 tras un cierre no limpio del sistema no depende del tamaño del sistema de archivos ni del número de archivos, sino del tamaño del journal (diario), utilizado para mantener la consistencia en el sistema.
- Por defecto, la recuperación del tamaño del "journal" tarda alrededor de un segundo, según la velocidad del hardware.

EXT4

- Ext4 nació como una serie de extensiones compatibles para ext3 que solucionaban algunas limitaciones
- Luego evolucionó y a partir de un fork de ext3 terminó siendo recomendado para adoptar dentro del kernel de Linux.
- Es compatible con ext2 y ext3
- Las principales mejoras son:
 - Soporte de volúmenes de hasta 1024 PiB
 - Soporte añadido de extent
 - Menor uso del CPU
 - Mejoras en la velocidad de lectura y escritura

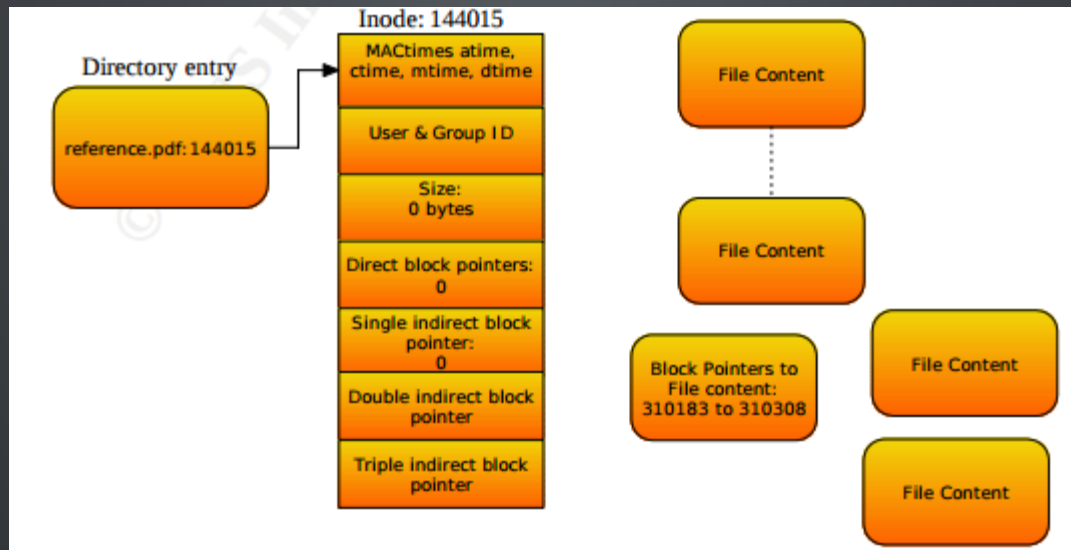
BORRADO DE ARCHIVOS EXT2

- Cuando un archivo es borrado, el inodo y los bloques pertenecientes son marcados como desalocados entonces el SO puede reusarlos cuando lo necesite pero básicamente toda la información sigue existiendo hasta que realmente se reutilice el espacio.



BORRADO DE ARCHIVOS EXT3/4

- Dentro del inodo el tamaño del archivo como los punteros a bloques son puestos en cero dejándonos sin manera de ubicarlos nuevamente.



BORRADO DE ARCHIVOS

- En EXT2 era relativamente sencillo recuperar los archivos borrados siempre y cuando no hubieran sido sobrescritos
- En EXT3/4 aunque los enlaces a los bloques y el tamaño del archivo hayan sido borrados aún existe una manera de recuperar ese archivo.
 - Ejemplo recurriendo al journalist: debugfs y ls -d

CHEATSHEET

- En el gráfico ejemplo hace mención a herramientas, muchas de los cuáles utilizaremos a continuación.

